

Haven — arkitektur

Pedagogisk genomgång av hur systemet är uppbyggt: Safe-konton, de smarta kontrakten, EOA-nycklarna, kodstrukturen och pengaflödena — plus förbättringsförslag.

Agent-first wallet-infrastruktur (non-custodial).

Grundad i den faktiska koden (backend/lib, chains.ts, safe-deployer, allowance-module, sweep).

2026-07-01

1. Grundidén: tre nycklar, ett Safe-konto

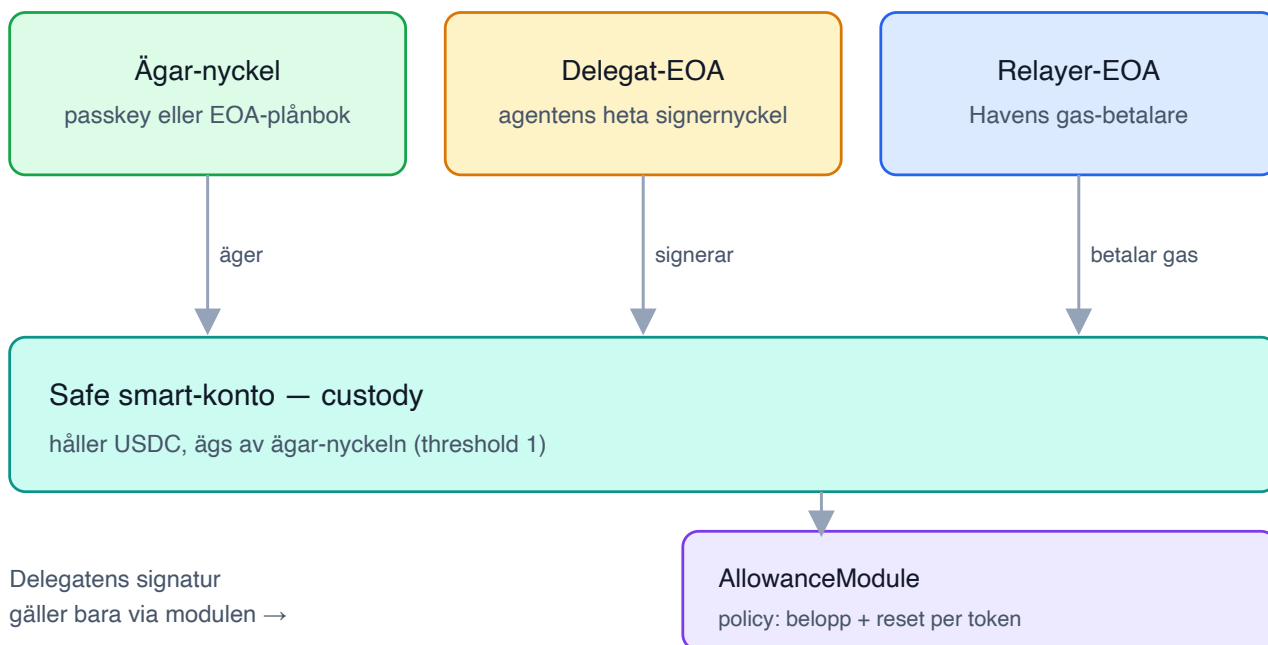
Allt i Haven kretsar kring att separera **tre olika nycklar** som var för sig är nästan maktlösa. Det är detta som gör systemet non-custodialt: ingen enskild nyckel — särskilt inte någon Haven håller — kan flytta användarens pengar fritt.

Ägar-nyckel (owner) — en passkey (WebAuthn, nyckeln i enhetens secure enclave) eller en vanlig EOA-plånbok (t.ex. MetaMask). Den **äger Safe-kontot**: lägger till/tar bort ägare, aktiverar moduler och godkänner betalningar som ligger över agentens allowance. Haven har den aldrig.

Delegat-EOA — agentens heta signeringsnyckel, **genererad klient-sidan** (i SDK/signer/connect). Haven lagrar bara *adressen*, aldrig privatnyckeln (verifierat: ingen server-side keygen i backend). Den signerar auktoriseringar men **håller aldrig ETH** och kan bara röra pengar *inom* den allowance ägaren beviljat.

Relayer-EOA — Havens egen driftsnyckel (`RELAYER_PRIVATE_KEY_<chainId>`). Den **betalar gas och submittar** allt on-chain, men **håller bara gas** — aldrig allowance, aldrig custody. Även om den komprometteras kan en angripare bara slösa gas.

Tre nycklar och ett Safe-konto



Figur 1 — De tre nycklarna och deras enda befogenheter. Modulen är policygränsen.

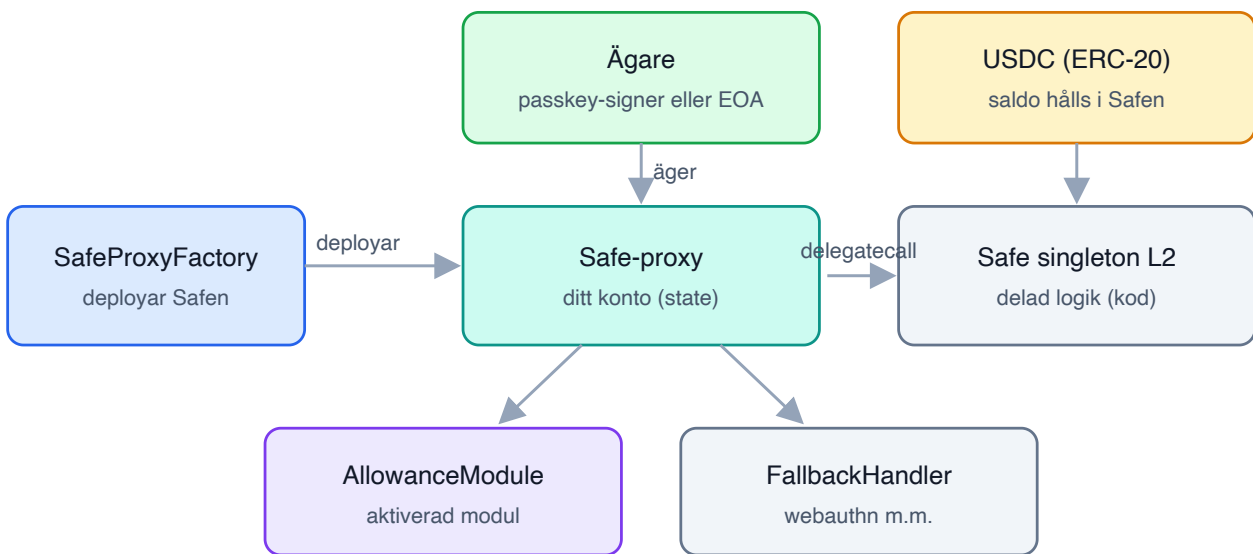
2. De smarta kontrakten on-chain

En viktig sak: Haven har **ingen egen Solidity-kod** (verifierat — inga `.sol`-filer, ingen hardhat/foundry). Systemet använder de *kanoniska* Safe-kontrakten plus Safe:s officiella `AllowanceModule`. Backend pratar med dem via direkta `ethers.js`-anrop; kedjeregistret ligger i `chains.ts`.

Per kedja (Base 8453, Base Sepolia 84532, Gnosis 100) finns dessa delar:

- **Safe-proxy** — *ditt konto*. Ett litet proxy-kontrakt som lagrar all state (ägare, saldon, aktiverade moduler) men `delegatecall`:ar in logiken från singletonen.
- **Safe singleton (L2)** — den delade implementationen (koden) alla proxies pekar på, så deploy blir billig.
- **SafeProxyFactory** — det relayern anropar (`createProxyWithNonce`) med `setup([owner], 1, ...)`.
- **AllowanceModule** — den aktiverade modulen som är själva policyn: per (Safe, delegat, token) håller den belopp, spenderat, reset-period och nonce.
- **FallbackHandler + passkey**: `SafeWebAuthnSignerFactory` och en P-256-verifierare låter en passkey agera ägare (signer-adressen beräknas via CREATE2 i `passkey-signer.ts`).

On-chain-kontrakten (per kedja)



Kedjor: Base 8453 · Base Sepolia 84532 · Gnosis 100 — samma kontraktsuppsättning per kedja

Figur 2 — Kontraktsstacken. Proxyn håller state; singletonen håller koden; modulen håller policyn.

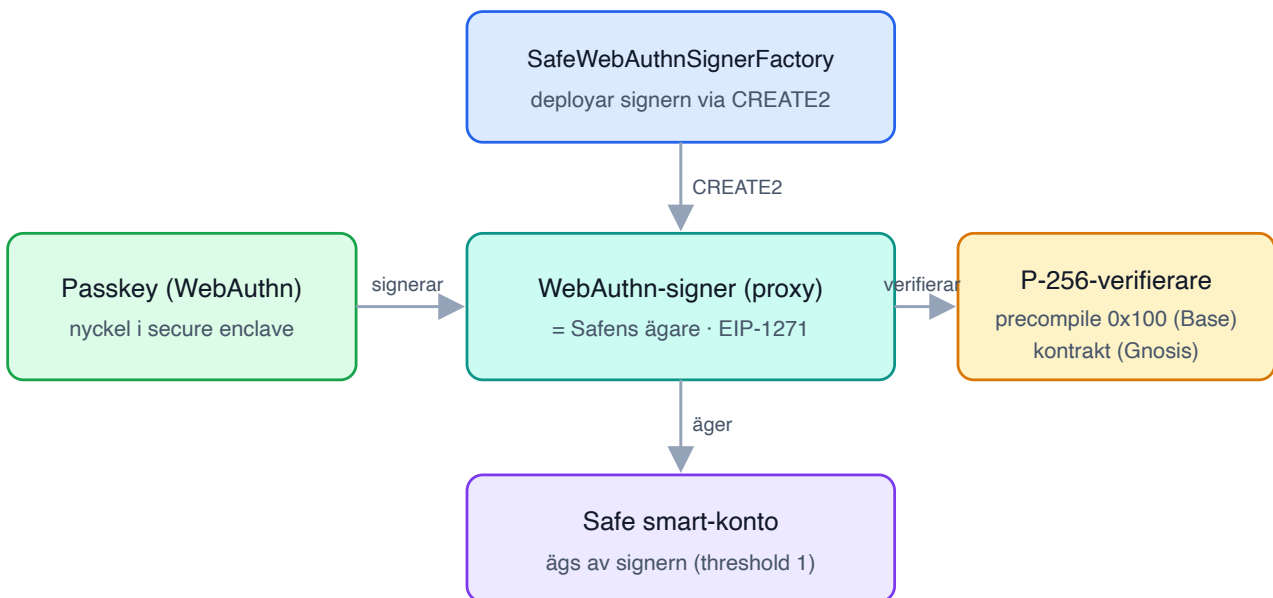
3. Passkey-lösningen on-chain

Passkey-ägarskap är flera egna on-chain-kontrakt — här zoomar vi in på "Ägare"- och "FallbackHandler"-rutorna från figur 2. En **passkey** är en WebAuthn-credential vars privatnyckel bor i enhetens secure enclave och **aldrig lämnar den**. Den signerar med kurvan **P-256 (secp256r1)** — inte secp256k1 som Ethereum använder. Ethereum kan alltså inte verifiera signaturen "gratis", och det är hela poängen med kontrakten:

- **SafeWebAuthnSignerFactory** (`0x1d31F259...` , samma kanoniska adress på alla tre kedjor) — deployar en liten **signer-proxy per passkey** via CREATE2.
- **WebAuthn-signer (proxy)** — det är *detta* kontrakt som är Safens ägare. En minimal proxy som `delegatecall` :ar en delad singleton (factoryn deployar den först, CREATE-nonce 1). Den implementerar **EIP-1271 isValidSignature** : tar en WebAuthn-assertion och verifierar P-256-signaturen.
- **P-256-verifierare** — här skiljer sig kedjorna: på **Base** är det den native **RIP-7212-precompilen** på `0x...0100` (billig, snabb); på **Gnosis** ett deployat verifierarkontrakt (`0x445a...`).

Eftersom signer-adressen är **deterministisk** (CREATE2 av passkeyns `x,y` + verifierare + singleton) kan Haven **räkna ut ägaradressen innan signern ens är deployad** och sätta den som Safens ägare direkt. Signern kan deployas lat, vid första användning. Fallback handlern (avsnittet ovan) ger Safens EIP-1271 på konto-nivå som pekar på ägaren.

Passkey-lösningen on-chain



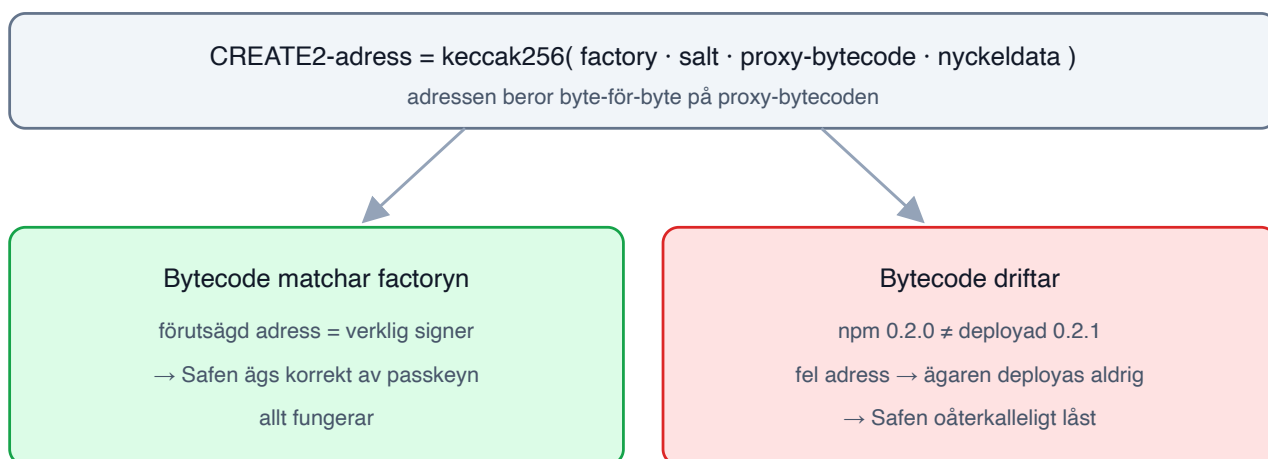
Figur 3 — Passkey-kontrakten. Factoryn CREATE2-deployar en signer-proxy som blir Safens ägare och verifierar P-256 via precompile (Base) eller kontrakt (Gnosis).

Adress-drift — ett tyst och farligt failure mode

CREATE2 ger en deterministisk adress: `keccak256(0xff · factory · salt · keccak256(initCode))`, där `initCode` = *proxy-bytecoden* + de ABI-kodade argumenten (`singleton`, `x`, `y`, `verifier`). Haven räknar ut adressen **off-chain på två ställen** — backend (`passkey-signer.ts`) och frontend (`safePasskeySigner.ts`) — och koden kräver att **båda ger exakt samma resultat**.

Buggen. För att förutsägelsen ska stämma måste proxy-bytecoden off-chain vara **byte-identisk** med den factoryn använder on-chain. Men `@safe-global/safe-passkey` på npm skeppar **v0.2.0**-bytecode medan den live-deployade factoryn är **v0.2.1** — olika inbäddad proxy-bytecode. Med npm-bytecoden skulle CREATE2 räkna fram en **annan adress** än factoryn faktiskt deployar → Safen får en ägare som aldrig existerar där → **medlen är låsta**. Därför hårdkodar `passkey-signer.ts` den långa hex-literalen, handplockad ur den live v0.2.1-factoryn, i stället för npm-bytecoden.

Varför passkey-adressen kan drifta



Figur 4 — Off-chain-förutsägelsen och on-chain-factoryn måste dela exakt samma proxy-bytecode, annars får kontot en spök-ägare.

Varför det är farligt. Inget felar vid deploy — Safen deployas, adressen räknas ut, allt ser grönt ut. Det är bara *fel*: kontot ägs av en spök-ägare, och felet är osynligt tills passkeyn ska användas (då kan det redan ligga pengar i Safen). Det är också skört: driften återinförs tyst om Safe deployar om factoryn (v0.2.2), en ny kedja använder en annan version, någon byter tillbaka till npm-bytecoden, eller backend- och frontend-literalerna glider isär.

Förbättring (skarpare än dagens kod-kommentar): gör förutsägelsen **CI-verifierad mot den riktiga factoryn** per kedja — deploya en signer på en fork/testnet och assert:a att `predictSafePasskeySignerAddress(x,y)` matchar den verkliga adressen; lägg ett paritetstest `backend-literal === frontend-literal`. Då blir en bytecode-drift ett rött CI-bygge i stället för ett brickat konto. Ännu bättre: läs proxy-bytecoden från factoryn i runtime om den exponerar den.

4. Kodstrukturen (monorepot)

Repot är ett npm-workspace-monorepo. Det centrala rastret: vissa paket kör **på agentens/kundens sida**, ett kör **Havens kontrollplan**, och de smarta kontrakten lever **on-chain**.

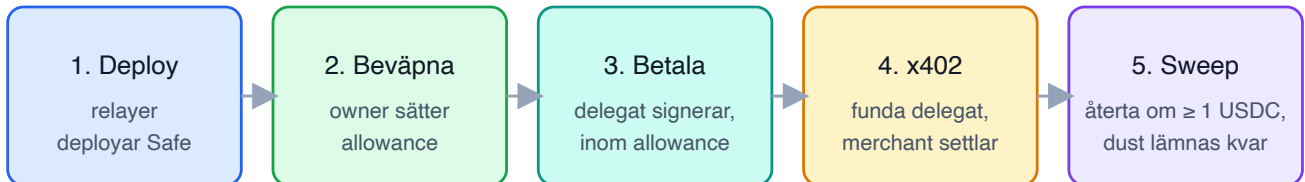
```
Haven-AI/ (npm workspaces)
├─ packages/backend/      ← HAVENS HJÄRNA (Fastify + Postgres)
│   └─ src/
│       ├── routes/      HTTP-ytan: /agents, /payments, machine-payments,
│       │               x402, safe-deploy, safe-exec
│       └─ lib/          on-chain-lagret (ethers.js, ingen protocol-kit):
│           ├── chains.ts    kedjeregister (RPC, kontraktsadresser, tokens)
│           ├── relay.ts     per-chain relay-signer (#640/#678)
│           ├── allowance-module.ts läser/exekverar AllowanceModule
│           ├── safe-deployer.ts createProxyWithNonce → deployar Safen
│           ├── safe-exec.ts   owner-tx (enable module, allowance, godkänn)
│           ├── passkey-signer.ts WebAuthn-ägarens signer-adress (CREATE2)
│           ├── sweep.ts      gasless EIP-3009-sweep (delegat → Safe)
│           └─ config.ts      relay-nycklar, served chains, sweep-cap
├─ packages/frontend/     Next.js-dashboard (onboarding, saldon, agenter)
│   └─ agent-/kund-sidan (publiceras på npm) –
├─ packages/sdk/          @haven_ai/sdk – agentklient: pay(), x402, sweep,
│                           håller delegateKey KLIENT-SIDAN
├─ packages/signer/       @haven_ai/signer – lokal delegat-signer
├─ packages/mcp/          @haven_ai/mcp – MCP-verktygsyta
├─ packages/mcp-server/   hostad MCP-server (deployas, ej npm)
├─ packages/connect/      @haven_ai/connect – `npx @haven_ai/connect` kopplaren
├─ packages/cli/          @haven_ai/cli
│   └─ test/demo –
├─ packages/demo-merchant-mcp/ demo-x402-handlare (för QA + demo)
└─ packages/qa-agent/     deterministisk money-flow-harness + seed
```

Poängen med uppdelningen: **privatnycklar korsar aldrig gränsen**. Delegat-nyckeln bor i sdk / signer / connect hos agenten; backend ser bara adresser och signaturer. Backend är det enda som håller en *drifts*-nyckel (relayern), och den kan bara betala gas.

5. Pengaflödena (livscykeln)

Fyra flöden räcker för att förstå hela systemet. Gemensamt för alla: **relayern betalar gas, delegaten signerar men håller aldrig ETH.**

Livscykeln — deploy till sweep



Relayern betalar gas i varje steg. Delegaten signerar men håller aldrig ETH.

Figur 5 — Livscykeln från konto-deploy till gasless sweep-återhämtning.

Vad som händer i varje steg

- 1. Deploy** — kunden skapar en passkey (eller kopplar en EOA). Relayern anropar `SafeProxyFactory.createProxyWithNonce` med `setup([owner], 1, ...)` → Safen finns on-chain, ägd av kunden, threshold 1, utan moduler. Kunden fyller Safen med USDC.
- 2. Beväpna** — ägaren signerar en Safe- `execTransaction` (via `safe-exec.ts`) som aktiverar `AllowanceModule` och sätter delegatens allowance (belopp + reset-period). Relayern submittar. Agenten får en API-nyckel (från Haven) + delegat-nyckeln (klient-sidan).
- 3. Betala** — agenten skickar en intent → backend läser on-chain-allowancen (`getTokenAllowance`). Ryms den → delegaten signerar en transfer-hash → relayern submittar `executeAllowanceTransfer` → USDC går Safe → mottagare. Över allowance → köas som pending för ägaren att godkänna. Ingenting exekveras utanför ramen.
- 4. x402** — agenten möter HTTP 402 → Haven fundar delegaten från Safen (en allowance-transfer Safe → delegat) → agenten signerar en standard x402 EIP-3009 -betalning från delegaten → merchantens facilitator settlar delegat → merchant. Delegaten håller kort het USDC här.
- 5. Sweep** — om merchanten verifierar men inte settlar blir USDC strandad på delegaten. Delegaten signerar en EIP-3009 `transferWithAuthorization` → relayern submittar gasless → USDC går delegat → Safe. Sweepen körs bara om det strandade beloppet är **minst 1 USDC** (ett golv, `SWEEP_MIN_USDC`); mindre "dust" lämnas kvar på delegaten eftersom gasen skulle överstiga värdet. Större belopp återtas normalt. (Dev sätter golvet till 0 så QA fortfarande testat en riktig sweep.)

6. Förbättringsförslag

Rangordnade efter värde. De flesta är *inte* säkerhetshål — non-custody-invarianten håller. De handlar om att minimera det enda heta fönstret, om drift och om policy-uttryckskraft. Varje punkt är spårad som GitHub-issue.

Hög prioritet

1. **Delegat-in-flight-exponeringen (epic #713)**. Den strukturellt svagaste punkten: när Safen fundat delegaten ligger beloppet på en het EOA *utanför all policy* tills merchanten settlar — en komprometterad delegat-nyckel kan stjäla det som är i transit. Mildringar: delegat-saldo-monitor + larm (#714, fångar även dust som ackumuleras under sweep-golvet), åtstramade `validBefore` -fönster på EIP-3009 (#715), enforced exakt-belopps-funding + reconciliation efter settle (#716). Strukturella fixen — Safen betalar merchanten direkt — utforskas i #452.
2. **Gas-abuse / relayer-tömning (#717)**. Haven sponsrar all gas; spam kan dränera relayern ("relayer unfunded" för alla). Per-agent gas-budget + rate-limits, saldolarm per kedja, gas-metrik per agent. Inte custody — availability och kostnad.
3. **Nonce-koordinering över replikor (#718)**. Allowance-nonce-koordinatören är in-process: skalas backend till >1 replika återkommer stale-nonce-racet tyst (preflighten håller det *säkert*, inte snabbt). Flytta koordineringen till delad state (Postgres-lås); utvärdera relayer-nyckelpool för throughput. Tills dess: skala vertikalt.
4. **Rikare policy än AllowanceModule (design-ADR #719)**. Idag är policyn enbart belopp + reset per token — ingen mottagar-allowlist, inga per-tx-tak, inga kategoriregler; modulen (v0.1.1) rymmer inte roadmapen. Beslutat spår (förslag): **session keys via Safe7579 + Smart Sessions** — delegat-nyckeln blir en session key med on-chain-policies (belopp + refill, allowlist, per-tx-tak, tidsgräns), migrerbar med en ägar-tx per konto. Guard-spåret förkastat (module-tx:er passerar inte guards på deployade v1.3/v1.4.1-singletons); Zodiac Roles förkastat (bryter gasless-delegaten). Pensionerar samtidigt #713/#717/#718 strukturellt — se rutan nedan.

Medel

5. **Threshold 1 / single owner**. För högre värden: stöd 2-av-N eller guardian/social recovery. Non-custody blir starkare med en återhämtningsväg om ägar-nyckeln tappas.
6. **Passkey-adress-drift (se avsnitt 3)**. Den handkopierade proxy-bytecoden är skör och ett tyst, oåterkalleligt failure mode. CI-verifiera `predictSafePasskeySignerAddress` mot den riktiga factoryn per kedja + paritetstesta backend- mot frontend-literalen, så en drift blir ett rött bygge i stället för ett brickat konto.

Lägre / städning

7. **Domänuppdelning i backend/lib**. Chain-ytan (relayer, allowance, sweep, safe-*) ligger blandad med bokföring (fortnox, sie, vat, ledger). Dela i tydliga domäner (`chain/` , `payments/` , `accounting/`) för en skarpare kognitiv karta. Kosmetiskt.

ERC-4337 — reviderad hållning (ADR #719). Att migrera *för skalskäl* är fortsatt uppskjutet: bundlerberoenden och ny rörlig komplexitet väger tungt i ett CASP-känsligt läge. Men policy-roadmapen är den legitima triggern — session keys (Safe7579 + Smart Sessions) vänder 4337-komplexiteten från kostnad till det fordon som pensionerar policy-frågan, delegat-in-flight (#713, via EIP-1271-x402 i #452), gas-sponsringen (#717, paymaster med per-agent-budget) och nonce-flaskhalsen (#718, bundlers) i ett drag. Vägen är stegvis: pilot på Base Sepolia → en ägar-tx per konto → AllowanceModule fasas ut. Ingen big-bang.